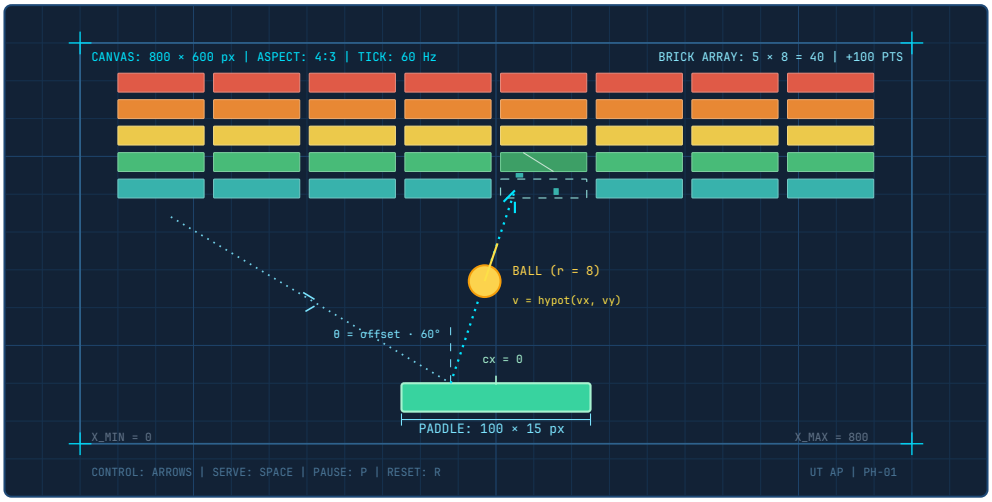
**دانشگاه تهران**
دانشکدگان فنی

## برنامه‌سازی پیشرفته

فاز پروژه

# ۱ه هستهٔ بازی Breakout

اهداف، نیازمندی‌های فنی، طراحی شی‌گرا و معیارهای ارزیابی




«در سال ۱۹۷۶، نولان باشنل در شرکت آتاری از استیو جابز خواست بازی Breakout را با کمترین تعداد تراشهٔ سخت‌افزاری بسازد. جابز پروژه را به استیو وازنیاک سپرد؛ وازنیاک طی ۴ شبانه‌روز کار بی‌وقفه، مدار بازی را تنها با ۴۴ تراشه پیاده کرد. ابتکارات وازنیاک در بهینه‌سازی مدارهای گرافیکی این بازی، مستقیماً الهام‌بخش او در طراحی معماری نمایش تصویر و رنگ در رایانهٔ انقلابی Apple II شد.»

**پرامپت پیشنهادی برای تولید تصویر هنری کاور**

*Technical blueprint drawing of classic 1976 Breakout arcade game, showing green phosphor wireframe paddle, mathematical ball deflection angles, and brick array grid on deep navy blueprint paper, clean line art, precise dimension labels, engineering schematic style.*

# فهرست مطالب

هدف و محدودهٔ آموزشی ..... 1
قواعد و سازوکار پایهٔ بازی ..... 1
معماری سیستم و تفکیک مسئولیت‌ها (MVC) ..... 2
مسئولیت کلاس‌ها و کپسوله‌سازی داده‌ها ..... 2
آینده‌نگری معماری: قرارداد ذخیره‌سازی داده ..... 2
دستگاه مختصات و هندسهٔ بازی ..... 3
بازتاب زاویه‌دار قطعه‌ای از سکو (Segmented Deflection) ..... 3
برخورد جعبه‌های مرزی (AABB) و تالار خطاهای متداول ..... 3
ماشین وضعیت و چرخهٔ حیات بازی (State Machine) ..... 4
گام‌های پنج‌گانهٔ پیاده‌سازی گام‌به‌گام (Milestones) ..... 4
بخش‌های اختیاری و امتیازی (Bonus Features) ..... 4
ساختار استاندارد مخزن گیت و فایل‌ها ..... 5
جدول مصوب ارزیابی و بارم‌بندی (۱۰۰ نمره) ..... 5
ضریب ارزیابی در جلسهٔ دفاع شفاهی (Defense Multiplier) ..... 5

## هدف و محدودهٔ آموزشی

هدف بنیادین این فاز، تمرین مدل‌سازی شی‌گرا با کلاس‌های مستقل جاوا، رعایت دقیق اصل تک‌مسئولیتی (Single Responsibility Principle)، کپسوله‌سازی کامل وضعیت درونی اشیا و تفکیک لایهٔ منطق محاسباتی از نمایش گرافیکی است. این پروژه بر پایهٔ مفاهیم آموزش‌داده‌شده تا چندریختی (Polymorphism) و معماری مبتنی بر رخداد (Event-Driven Architecture) بنا شده و ساختار انتزاعی آن بستر لازم را برای ورود به مبحث مدیریت فایل (File I/O) در فاز بعدی فراهم می‌سازد.

تمرکز این فاز بر ساختار تمیز، پایداری کد و آزمون‌پذیری اجزاست؛ رابط کاربری پیچیده، جلوه‌های بصری و منوهای تودرتو **الزامی نیستند**. خروجی باید بدون خطا، قابل‌اجرا و آمادهٔ گسترش باشد (بارم فاز اول: ۲ نمره).

## قواعد و سازوکار پایهٔ بازی

رفتارهای زیر به‌صورت قطعی و الزامی در چرخهٔ گیم‌پلی بازی پیاده‌سازی می‌شوند:

| سازوکار گیم‌پلی | مؤلفه / کلید | رفتار و قانون مورد انتظار در پیاده‌سازی |
|---|---|---|
| سیستم جان (Lives) | نمایشگر HUD (۳ جان) | آغاز با ۳ جان؛ خروج توپ از کف بوم موجب کسر ۱ جان می‌شود. نمایش وضعیت با ۳ قلب یا شمارنده در HUD. |
| شمارندهٔ امتیاز (Score) | نمایشگر HUD (+100) | تخریب هر آجر ۱۰۰ امتیاز به بازیکن می‌افزاید (بیشینهٔ مجموع: ۴۰۰۰ امتیاز معادل ۴۰ آجر). |
| پرتاب به درخواست بازیکن | کلید فاصله (Space) | در شروع و پس از کسر جان، توپ روی سکو ثابت (Attached) است؛ پرتاب تنها با فشردن Space آغاز می‌شود. |
| توقف موقت (Pause) | کلید P | توقف آنی حلقهٔ فیزیک و نمایش برچسب توقف؛ فشردن مجدد، بازی را بدون اختلال زمانی ادامه می‌دهد. |
| شروع مجدد سریع (Restart) | کلید R | بازنشانی آنی در هر لحظه: بازسازی ۴۰ آجر، بازگشت به ۳ جان، صفر شدن امتیاز و استقرار مجدد توپ روی سکو. |
| شرایط پایان (End States) | VICTORY / OVER | پیروزی با تخریب تمام ۴۰ آجر، و شکست قطعی با اتمام هر ۳ جان رخ داده و پیام وضعیت نمایان می‌شود. |

# معماری سیستم و تفکیک مسئولیت‌ها (MVC)

در پیاده‌سازی پروژه‌های تعاملی، بزرگ‌ترین خطای رایج تجمع تمام منطق بازی، شنوندگان رخداد، حلقه‌های زمانی و اشکال گرافیکی درون یک کلاس زیرمجموعهٔ `GraphicsProgram` است؛ الگویی نامناسب که به ساخت یک شیء همه‌کاره (God Class)، ناپایدار و غیرقابل آزمون منجر می‌شود. برای دستیابی به یک معماری اصولی و تفکیک مسئولیت‌ها (SoC)، سیستم به سه لایهٔ اصلی تفکیک می‌شود:

### نمایشگر (View)

`BreakoutCanvas`

- پوستهٔ بوم و کپسوله‌سازی ACM
- ترسیم `GRect` و `GOval`
- نمایش HUD (امتیاز و ۳ قلب جان)
- لایهٔ پیام وضعیت (Overlay)

### کنترلر (Controller)

`GameEngine`

- حلقهٔ بازی با زمان‌بندی 60 FPS
- شنوندهٔ ورودی صفحه‌کلید (Key Events)
- توزیع‌کنندهٔ برخوردها (Dispatcher)
- هماهنگ‌سازی مدل و فرمان بازترسیم

### مدل دامنه (Model)

دامنهٔ خالص جاوا

- کلاس‌های `Ball`، `Paddle`، `Brick`
- وضعیت بازی (`GameState`)
- قرارداد رکورد (`HighScoreStorage`)
- بدون وابستگی به GUI و کاملاً آزمون‌پذیر

## مسئولیت کلاس‌ها و کپسوله‌سازی داده‌ها

| کلاس | فیلدهای کپسوله‌شده (private) | رفتارها و مسئولیت‌های کلیدی |
|---|---|---|
| `Ball` | `x, y, vx, vy, radius` | به‌روزرسانی مختصات با بردار سرعت (`update`)، بازتاب افقی و عمودی (`bounceX, bounceY`) و استخراج جعبهٔ مرزی. |
| `Paddle` | `x, y, width, height, speed` | حرکت به چپ و راست بر اساس ورودی، و مهار موقعیت در محدودهٔ بوم (`clampToScreen`) جهت جلوگیری از خروج. |
| `Brick` | `x, y, width, height, durability, destroyed` | ثبت برخورد گلوله (`hit`)، کاهش مقاومت، تعیین امتیاز اعطایی (`getPoints`) و گزارش تخریب کامل شیء. |
| `GameEngine` | `ball, paddle, bricks, state, storage` | مدیریت تیک‌های زمانی 60 هرتز، پردازش ورودی‌ها، تشخیص برخوردها، گذار وضعیت و فرمان رندر به بوم. |

# آینده‌نگری معماری: قرارداد ذخیره‌سازی داده

یکی از اصول کلیدی طراحی شی‌گرا، وابستگی به انتزاع‌ها به‌جای پیاده‌سازی‌های انضمامی است. بدون ورود به پیچیدگی اصطلاحات الگوهای طراحی، تعریف یک رابط (Interface) ساده به ما اجازه می‌دهد ثبت بیشترین امتیاز (High Score) را بدون دستکاری هستهٔ بازی تغییر دهیم:

```
public interface HighScoreStorage {
    int loadHighScore();
    void saveHighScore(int score);
}
```

- **پیاده‌سازی فوری (`InMemoryScoreStorage`):** ذخیرهٔ بالاترین امتیاز در حافظهٔ موقت (RAM) طی اجرای جاری برنامه.
- **پیاده‌سازی آتی (`FileScoreStorage`):** خواندن و نوشتن رکورد در فایل متنی پس از آموزش مبحث File I/O.
- **چندریختی بدون بازنویسی:** موتور بازی (`GameEngine`) تنها با این رابط کار می‌کند و نوع ذخیره‌ساز در زمان ساخت به آن تزریق می‌شود؛ در نتیجه گذار به فاز بعدی نیازمند کوچک‌ترین تغییری در کدهای موتور نخواهد بود.

# دستگاه مختصات و هندسهٔ بازی

در محیط گرافیکی کامپیوتر، جهت‌گیری محورهای مختصات با دستگاه دکارتی رایج در ریاضیات تفاوت دارد: نقطهٔ مبدا $(0,0)$ در گوشهٔ بالا-چپ قرار دارد، محور $+X$ به سمت راست و محور $+Y$ به سمت پایین افزایش می‌یابد. در نتیجه علامت مؤلفهٔ سرعت عمودی نقشی حیاتی در رفتار توپ دارد:

### مختصات بوم و بردار سرعت

$(0,0) \longrightarrow +X$ (حرکت به راست)
$+Y \downarrow$ (افزایش عمق به سمت کف)

**قواعد جهت حرکت:**

- $v_y > 0$: سقوط رو به پایین
- $v_y < 0$: اوج‌گیری رو به بالا (سمت آجرها)

| مؤلفه | ابعاد استاندارد | توضیحات و رفتار |
|---|---|---|
| بوم (Canvas) | $600 \times 800$ پیکسل | محیط پایدار با نرخ به‌روزرسانی 60 FPS |
| سکو (Paddle) | $15 \times 100$ پیکسل | فاصلهٔ 40 پیکسلی از کف، مهار کامل در افق |
| توپ (Ball) | شعاع 8 (قطر 16) | آغاز حرکت با زاویهٔ روبه‌بالا و تندی ثابت |
| آجرها (Bricks) | ۵ سطر در ۸ ستون (40) | ابعاد $25 \times 85$ با فاصلهٔ 6 پیکسل |

# بازتاب زاویه‌دار قطعه‌ای از سکو (Segmented Deflection)

اگر توپ همواره با بازتاب آینه‌ای از سکو کمانه کند، بازی پس از چند ضربه وارد یک چرخهٔ تکراری و خسته‌کنندهٔ ۴۵ درجه می‌شود. با بازتاب زاویه‌دار، زاویهٔ پرتاب بر اساس فاصلهٔ برخورد از مرکز سکو کنترل‌پذیر می‌گردد:

- **انحراف نسبی:** $d = \frac{x_{\text{ball}} - x_{\text{paddle_center}}}{\frac{W_{\text{paddle}}}{2}} \in [-1.0, +1.0]$
- **زاویهٔ بیشینه:** $\theta = d \times 60^\circ$
- **تندی پایسته:** $v = \sqrt{v_x^2 + v_y^2}$
- **سرعت جدید:** $v_{x'} = v\sin(\theta)$ و $v_{y'} = -v\cos(\theta)$

مقدار $v_{y'}$ همواره منفی است تا بازتاب قطعاً رو به بالا باشد.

```
// Calculate hit offset relative to paddle
center (-1.0 to +1.0)
double paddleCenter = paddle.getX() +
paddle.getWidth() / 2.0;
double ballCenter = ball.getX() +
ball.getRadius();
double offset = (ballCenter - paddleCenter) /
(paddle.getWidth() / 2.0);
offset = Math.max(-1.0, Math.min(1.0, offset));

double maxAngle = Math.toRadians(60.0);
double angle = offset * maxAngle;
double speed = Math.hypot(ball.getVx(),
ball.getVy());

ball.setVx(speed * Math.sin(angle));
ball.setVy(-Math.abs(speed * Math.cos(angle)));
```

# برخورد جعبه‌های مرزی (AABB) و تالار خطاهای متداول

برای تشخیص برخورد دایره با مستطیل، نزدیک‌ترین نقطهٔ مستطیل به مرکز دایره محاسبه $(c_x = \text{clamp}(x, x_r, x_r + w)$ و $c_y = \text{clamp}(y, y_r, y_r + h))$ و فاصلهٔ آن با مرکز دایره مقایسه می‌شود $(d^2 \le r^2)$.

### تالار خطاهای متداول در پیاده‌سازی فیزیک

۱. **باگ سکوی چسبنده (Sticky Paddle):** معکوس‌کردن صرف علامت $v_y$ بدون خارج‌کردن توپ، باعث گیر افتادن و لرزش مداوم درون سکو می‌شود. هنگام برخورد باید فوراً مختصات توپ روی `ball.setY(paddle.getY() - ball.getRadius())` بازنشانی شود.

۲. **خطای برش در تقسیم صحیح (Integer Truncation):** استفاده از عملوند عدد صحیح در فرمول انحراف (`width / 2`) مقدار اعشاری را حذف کرده و نتیجه همواره صفر خواهد شد؛ حتماً از مخرج اعشاری `2.0` و متغیرهای `double` استفاده کنید.

۳. **تونل‌زنی در سرعت بالا (High-Speed Tunneling):** اگر گام جابه‌جایی توپ در یک فریم از ضخامت آجر یا سکو بیشتر شود، توپ بدون برخورد از مانع عبور می‌کند. سرعت باید مقداری محدود و متناسب با ابعاد اشکال باشد.

# ماشین وضعیت و چرخهٔ حیات بازی (State Machine)

مدیریت تعاملات و جریان بازی نیازمند یک ماشین حالت متناهی (FSM) شفاف است تا رفتار موتور در واکنش به رخدادهای صفحه‌کلید، از دست رفتن جان یا اتمام آجرها به شکلی قطعی و بدون تداخل هدایت شود:




| وضعیت | شرح رفتار و گذارهای مجاز |
|---|---|
| READY | حالت آغازین بازی یا پس از هربار سوختن جان؛ توپ به مرکز سکو چسبیده و همگام با آن جابه‌جا می‌شود. فشردن کلید Space بردار سرعت اولیه را فعال کرده و بازی را وارد وضعیت RUNNING می‌کند. |
| RUNNING | وضعیت اصلی شبیه‌سازی با نرخ نوسازی ۶۰ هرتز؛ جابه‌جایی سکو با کلیدهای جهت‌نما، محاسبات فیزیک توپ، بازتاب از دیواره‌ها و تخریب آجرها در این حالت فعال است. |
| PAUSED | با فشردن کلید P شبیه‌سازی فیزیک متوقف شده و یک پردهٔ نیمه‌شفاف با نشان PAUSED روی بوم ظاهر می‌شود. فشردن مجدد P بازی را ادامه می‌دهد. |
| GAME_OVER | با سقوط توپ از کف و اتمام ۳ جان رخ می‌دهد؛ شبیه‌سازی متوقف، پیام باخت نمایش، بالاترین امتیاز در منبع ذخیره‌سازی ثبت و کلید R بازی را از نو راه‌اندازی می‌کند. |
| VICTORY | با تخریب هر ۴۰ آجر فعال می‌شود؛ پیام پیروزی نمایش داده شده، رکورد نهایی ثبت و کلید R زمین را برای بازی جدید بازنشانی می‌کند. |

# گام‌های پنج‌گانهٔ پیاده‌سازی گام‌به‌گام (Milestones)

برای جلوگیری از سردرگمی در توسعه، فرآیند پیاده‌سازی به ۵ فاز مشخص با معیارهای اعتبارسنجی مستقل تفکیک شده است:

۱. **گام اول (پنجره، بوم و سکوی مهارشده):** راه‌اندازی پنجره به ابعاد $800 \times 600$، ساخت کلاس سکو (Paddle) و کنترل با کلیدهای چپ و راست.
معیار پذیرش: سکو با کلیدهای جهت‌نما نرم حرکت کند و تابع clampToScreen مانع خروج آن از مرزهای چپ و راست شود.

۲. **گام دوم (حرکت توپ، بازتاب دیواره‌ها و پرتاب دستی):** پیاده‌سازی کلاس توپ (Ball)، بازتاب از ۳ دیواره و حالت سکون روی سکو.
معیار پذیرش: توپ در وضعیت آماده‌باش همراه با سکو حرکت کند و پس از فشردن Space، با زاویه به سمت بالا پرتاب شود.

۳. **گام سوم (بازتاب زاویه‌دار از سکو و چرخهٔ ۳ جان):** پیاده‌سازی فرمول انحراف زاویه‌دار از سکو ($[-60^\circ, +60^\circ]$) و مدیریت قلب‌های جان در HUD.
معیار پذیرش: با عبور توپ از کف صفحه، یک جان کسر و یک قلب حذف شود؛ توپ مجدداً در وضعیت آماده‌باش روی سکو قرار گیرد.

۴. **گام چهارم (چیدمان ماتریس آجرها، برخورد AABB و ثبت امتیاز):** ترسیم ۴۰ آجر ($8 \times 5$)، تشخیص برخورد و افزایش امتیاز ($+۱۰۰$).
معیار پذیرش: برخورد با هر آجر موجب حذف فوری آن و بازتاب صحیح توپ شود؛ بیشترین امتیاز در شیء InMemoryScoreStorage ذخیره گردد.

۵. **گام پنجم (ماشین وضعیت جامع، پرده‌های نمایشی و شروع مجدد):** یکپارچه‌سازی متغیر GameState، پرده‌های گرافیکی و شروع مجدد با کلید R.
معیار پذیرش: باخت با ۳ خطا، برد با شکستن ۴۰ آجر، توقف موقت با P و بازنشانی کامل بازی با کلید R بدون خطای زمان اجرا.

# بخش‌های اختیاری و امتیازی (Bonus Features)

- **آجرهای مقاوم با دو جان (10+ نمره):** تعریف نوعی آجر ویژه در سطرهای بالایی که برای تخریب به 2 برخورد پیاپی نیاز دارند. ضربهٔ نخست موجب تغییر رنگ یا نمایش ترک‌خوردگی دیداری شده و ضربهٔ دوم آجر را از بین می‌برد ($+۲۰۰$ امتیاز).
- **تعویض پویای روش ذخیره‌سازی داده (5+ نمره):** امکان جابه‌جایی بلادرنگ نوع ذخیره‌ساز رکورد در کنترلر بین حافظهٔ موقت (InMemoryScoreStorage) و فایل دیسک با فشردن کلید درون بازی (بدون نیاز به راه‌اندازی مجدد برنامه).

## ساختار استاندارد مخزن گیت و فایل‌ها

پروژه باید مطابق با ساختار بسته‌بندی استاندارد جاوا و اصول تفکیک لایه‌ها سازمان‌دهی شود. قرارگیری فایل‌ها در پوشه‌های اختصاصی زیر الزامی است:

```
breakout/
├── src/
│   ├── model/          # Ball, Paddle, Brick, GameState,
│   │                   # HighScoreStorage, InMemoryScoreStorage
│   ├── view/           # BreakoutCanvas, HUD (Score & Hearts), Overlays
│   ├── controller/     # GameEngine, Main
│   └── util/           # GameConstants
├── lib/                # acm.jar
└── README.md
```

- **کامیت‌های اتمیک و معنادار:** فرآیند توسعه باید گام‌به‌گام در مخزن ثبت شود. به ازای هر یک از گام‌های پنج‌گانهٔ پیاده‌سازی، حداقل یک کامیت مستقل با پیام استاندارد ثبت کنید؛ برای نمونه `feat(paddle): implement movement and clamp` یا اصلاحیهٔ `fix(ball): solve sticky paddle issue` در انتهای هر مرحله.
- **پرهیز از کامیت تجمیعی:** تحویل کل پروژه در قالب یک کامیت واحد در پایان کار، مغایر با اصول مهندسی نرم‌افزار بوده و نمرهٔ بخش کیفیت کد را کاهش می‌دهد.
- **فایل راهنما (`README.md`):** شامل مشخصات کامل، نسخهٔ جاوا، نحوهٔ کامپایل و اجرای برنامه و توضیحات مختصر معماری باشد.

## جدول مصوب ارزیابی و بارم‌بندی (۱۰۰ نمره)

| معیار ارزیابی | مؤلفه‌ها و جزئیات بررسی فنی | بارم |
|---|---|---|
| درستی عملکرد و حس بازی | حرکت روان ۶۰ هرتز، پرتاب دستی با کلید Space، بازتاب زاویه‌دار از سکو، بازتاب الاستیک از ۳ دیواره، تخریب دقیق آجرها با الگوریتم AABB، چرخهٔ ۳ جان و ماشین وضعیت جامع. | ۴۰ نمره |
| معماری و طراحی شی‌گرا | تفکیک دقیق سه‌لایه (MVC)، کپسوله‌سازی کامل فیلدها با سطح دسترسی `private`، تک‌مسئولیتی کلاس‌ها، طراحی و بکارگیری رابط قرارداد ذخیره‌سازی (`HighScoreStorage`). | ۳۵ نمره |
| کیفیت کد و مهندسی نرم‌افزار | پرهیز کامل از اعداد جادویی و تجمیع ثابت‌ها در کلاس `GameConstants`، نام‌گذاری گویای متغیرها و متدها، ساختار استاندارد بسته‌ها و مخزن گیت با کامیت‌های اتمیک. | ۱۵ نمره |
| تسلط در دفاع و پاسخگویی | شرح و اثبات ریاضیات زاویهٔ بازتاب، تشریح دقیق منطق برخوردها، توجیه ساختار کلاس‌های مدل و پاسخ مستدل به پرسش‌های مفهومی در جلسهٔ دفاع حضوری. | ۱۰ نمره |
| **مجموع کل** | **ارزیابی جامع فاز اول (معادل ۲ نمره از ۲۰ نمرهٔ کل پایانی درس)** | **۱۰۰ نمره** |

## ضریب ارزیابی در جلسهٔ دفاع شفاهی (Defense Multiplier)

$$\text{نمرهٔ نهایی} = \text{نمرهٔ فنی} \times \alpha, \quad \alpha \in [0.5, 1.0]$$

نمرهٔ ثبت‌شده در کارنامه حاصل‌ضرب نمرهٔ فنی پروژه در ضریب تسلط دانشجو در جلسهٔ مصاحبهٔ حضوری ($\alpha$) است:

- **تسلط کامل بر کد و تشریح منطق:** در صورت تسلط دانشجو بر تمام خطوط برنامه، پاسخ شفاف به سوالات فنی و درک عمیق محاسبات هندسی و شی‌گرایی، ضریب کامل ($\alpha = 1.0$) اعمال شده و 100% نمرهٔ فنی کسب‌شده حفظ می‌شود.
- **عدم تسلط، ناتوانی در توضیح یا کد مشکوک به تقلب:** در صورت ناتوانی در تحلیل توابع، ابهام در نحوهٔ عملکرد بخش‌ها یا ناتوانی در اعمال تغییر کوچک درخواستی در جلسهٔ دفاع، ضریب دفاع کاهش یافته ($\alpha \in [0.5, 0.99]$) و نمره به صورت خطی تا سقف ۵۰٪ جریمه کسر می‌گردد ($\alpha = 0.5$).

### تحویل پروژه و اخلاق آکادمیک

- **مهلت و بستر تحویل:** ثبت لینک مخزن گیت‌هاب (GitHub) پروژه در سامانهٔ کوئرا (Quera) تا پیش از موعد مقرر الزامی است. مخزن باید عمومی یا دارای دسترسی برای دستیاران آموزشی درس باشد.
- **هشدار اصالت کد:** استفاده از ابزارهای هوش مصنوعی بدون درک عمیق خط‌به‌خط اکیداً ممنوع است. هر کدی که توسط دانشجو در مصاحبه قابل توجیه نباشد مشمول جریمهٔ دفاع ($\alpha = 0.5$) خواهد شد. کارهای دارای شباهت ساختاری غیرمجاز به کمیتهٔ انضباطی ارجاع می‌شوند.